

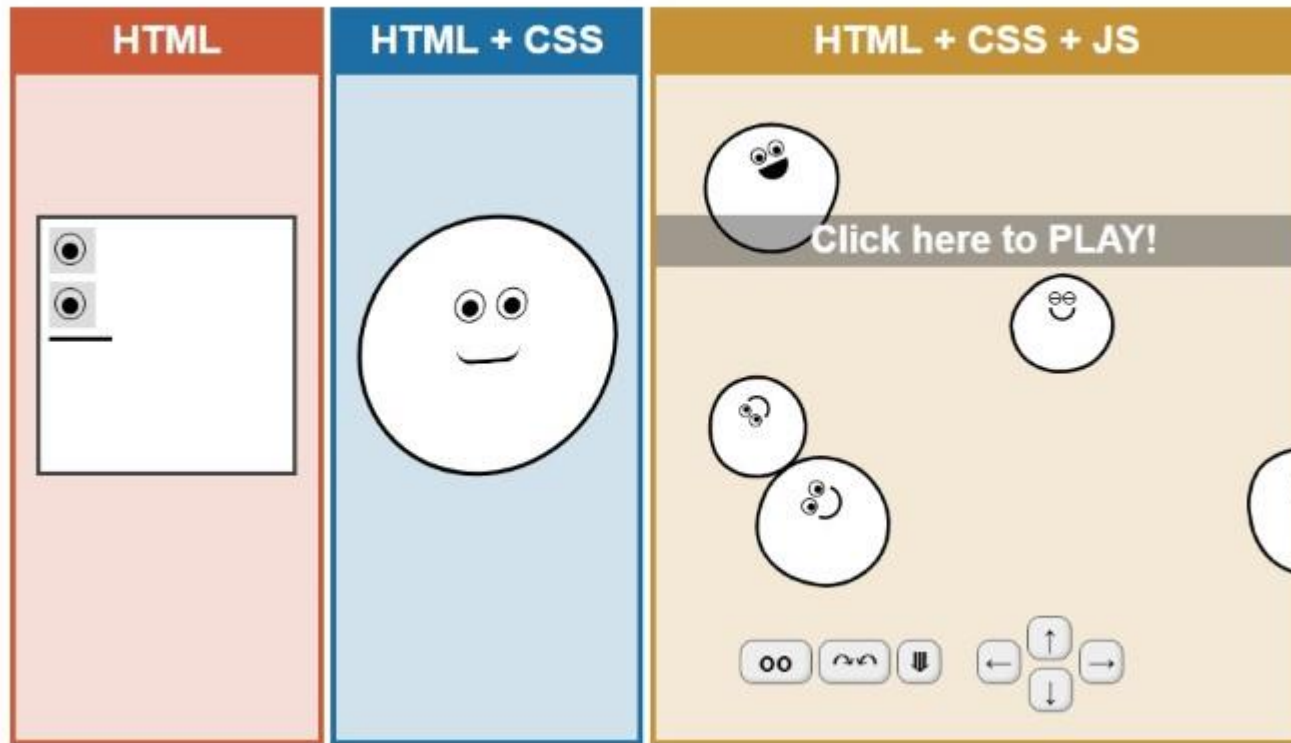
Programación Estática y Laboratorio Web

Javascript

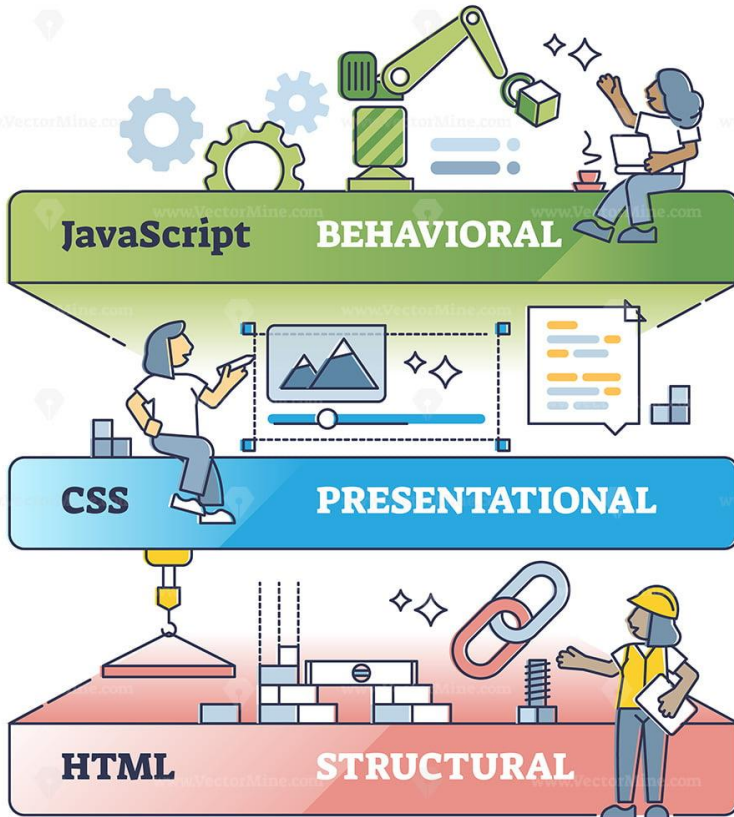


Universidad Nacional del Comahue
Facultad de Informática

[¿Se acuerdan?]



[Otras]



e!



HTML



HTML + CSS



HTML + CSS + JavaScript

[Javascript: funciones]

- Funciones
- El concepto es el mismo de todos los lenguajes.
- El valor devuelto es con la palabra reservada “return”

```
function saludar(nombre) {  
  console.log("Hola " + nombre);  
}  
  
saludar("María");  
  
function sumar(a, b) {  
  const resultado = a + b;  
  console.log(resultado);  
}  
  
sumar(5, 3);  
  
function sumar(a, b) {  
  return a + b;  
}  
  
const resultado = sumar(5, 3);  
  
console.log(resultado);
```

JavaScript: sentencia condicional

■ if else

```
if (condicion) {  
  // código si la condición  
  es verdadera  
} else {  
  // código si la condición  
  es falsa  
}
```

```
const edad = 20;  
  
if (edad >= 18) {  
  console.log("Puede ingresar");  
} else {  
  console.log("No puede ingresar");  
}  
  
function estaAprobado(nota) {  
  if (nota >= 6) {  
    return true;  
  } else {  
    return false;  
  }  
}  
  
const aprobado = estaAprobado(7);  
  
console.log(aprobado);
```

JavaScript: switch

■ Chequeo por múltiples condiciones

```
switch (valor) {  
  case opcion1:  
    // código si valor coincide con opcion1  
    break;  
  
  case opcion2:  
    // código si valor coincide con opcion2  
    break;  
  
  default:  
    // código si no coincide con ningún case  
    break;  
}
```

```
const dia = 3;  
  
switch (dia) {  
  case 1:  
    console.log("Lunes");  
    break;  
  
  case 2:  
    console.log("Martes");  
    break;  
  
  case 3:  
    console.log("Miércoles");  
    break;  
  
  default:  
    console.log("Día no válido");  
    break;  
}
```

JavaScript: bucles

- for
- while
- do while

```
for (let i = 1; i <= 5; i++) {  
  console.log("Repetición " + i);  
}  
  
while (contador <= 5) {  
  console.log(contador);  
  contador++;  
}  
  
let numero = 1;  
  
do {  
  console.log(numero);  
  numero++;  
} while (numero <= 5);
```

JavaScript: map

- Una forma de bucle sumamente utilizada
- Muy compatible con una colección recibida de manera externa (API, consulta DB).
- map es un método de los arreglos.
- Es muy usado en frameworks JavaScript para transformar listas de datos en elementos de interfaz.

```
const nombres = ["Jonathan", "Joseph", "Jotaro"];

const saludos =
  nombres.map(function (nombre) {
    return "Hola " + nombre;
  });

console.log(saludos);

const numeros = [1, 2, 3, 4];

const dobles = numeros.map(function (numero) {
  return numero * 2;
});

console.log(dobles); // [2, 4, 6, 8]
```

- Cada elemento de la colección se recibe como parámetro

JavaScript: eventos

- ¿Cuándo se ejecuta el código javascript?
- Programación basada en EVENTOS.
- ¿Qué puede ser un evento?
 - Un click
 - Un mouse over / out / click / up
 - Presión de una tecla
 - Página cargando
 - Mover el mouse
 - Envío del formulario
 - Timers

[Javascript: eventos]

- Vinculamos elementos de HTML a eventos.
- En esa vinculación, indicamos al navegador un código a ejecutar.
- Por ejemplo, cuando se hace click a un botón
- Cuando se escribe en un input text.
- Cuando el cursor del mouse pasa por encima de cualquier objeto



JavaScript: eventos

- Forma original de vincular eventos era con atributos de la etiqueta.
- Algunos de los más usados:
 - onclick: se ejecuta cuando el usuario hace clic sobre un elemento.
 - oninput: se ejecuta mientras el usuario escribe o cambia el valor de un campo.
 - onchange: se ejecuta cuando cambia el valor de un input, select o textarea.
 - onsubmit: se ejecuta cuando se intenta enviar un formulario.
 - onfocus: se ejecuta cuando un elemento recibe el foco.
 - onblur: se ejecuta cuando un elemento pierde el foco.
- Funcionan, pero no se recomiendan para código moderno.
- Dentro de cada uno de esos atributos, hay código javascript.

JavaScript: eventos



LEGACY

```
<button onclick="console.log('Hiciste clic')">
  Hacer clic
</button>

<input type="text" oninput="console.log('Está escribiendo')"></input>

<select onchange="console.log('Cambió la opción')">
  <option>HTML</option>
  <option>CSS</option>
  <option>JavaScript</option>
</select>

<form onsubmit="console.log('Se intentó enviar el formulario'); return false;">
  <input type="text" placeholder="Nombre">
  <button type="submit">Enviar</button>
</form>

<button onclick="console.log('Hiciste clic')">
  Hacer clic
</button>

<input type="text" onblur="console.log('Perdió el foco')"></input>

<body onload="console.log('Página cargada')"></body>
```

JavaScript: addEventListener

- La forma moderna.
- Se vinculan los objetos HTML con los eventos mediante la función “addEventListener”
- Lo vamos a terminar de comprender completamente cuando veamos DOM.
- Por ahora alcanza con saber que `document.querySelector` busca un elemento del HTML para poder trabajar con él desde JavaScript.”

JavaScript: addEventListener

- addEventListener permite asociar un evento a un elemento HTML.
- Primero seleccionamos el elemento de la página.
- Luego indicamos qué evento queremos escuchar.
- Finalmente indicamos qué función se ejecutará cuando ocurra ese evento.
- `objetoControl.addEventListener("evento", función)`
- La función puede recibir un parámetro, generalmente llamado `event`, con información sobre el evento ocurrido. Por ejemplo, permite saber qué elemento disparó el evento e información adicional.

Javascript: addEventListener

```
<button id="btnSaludar">Saludar</button>

<script>
  const boton = document.getElementById("btnSaludar");

  function saludar() { console.log("Hola"); }

  boton.addEventListener("click", saludar);
</script>
```

Al botón de id “btnSaludar” le vinculamos para el evento “click” la función “saludar”

```
<input id="teclado" type="text" placeholder="Presione una tecla">
<p id="tecla"></p>

<script>
  const teclado = document.getElementById("teclado");
  const tecla = document.getElementById("tecla");

  function mostrarTecla(event) {
    tecla.textContent = "Tecla presionada: " + event.key;
  }

  teclado.addEventListener("keydown", mostrarTecla);
</script>
```

addEventListener vincula el elemento de id “teclado” al evento “keydown”
Cuando se presiona una tecla en el input de id “teclado”, se ejecuta la función mostrarTecla que recibe por parámetro datos pertinentes al evento.

Javascript: addEventListener

```
<form id="formulario">
  <input id="nombreForm" type="text" placeholder="Nombre">
  <button type="submit">Enviar</button>
</form>

<p id="mensajeForm"></p>

<script>
  const formulario = document.getElementById("formulario");
  const nombreForm = document.getElementById("nombreForm");
  const mensajeForm = document.getElementById("mensajeForm");

  function validarFormulario(event) {
    event.preventDefault();

    if (nombreForm.value === "") {
      mensajeForm.textContent = "Debe ingresar un nombre.";
    } else {
      mensajeForm.textContent = "Formulario enviado correctamente.";
    }
  }

  formulario.addEventListener("submit", validarFormulario);
</script>
```

addEventListener vincula el evento "submit" del formulario a la función validar. El ejemplo es didáctico.

Javascript: addEventListener

```
<form id="formulario">
  <input id="nombreForm" type="text" placeholder="Nombre">
  <button type="submit">Enviar</button>
</form>

<p id="mensajeForm"></p>

<script>
  const formulario = document.getElementById("formulario");
  const nombreForm = document.getElementById("nombreForm");
  const mensajeForm = document.getElementById("mensajeForm");

  function validarFormulario(event) {
    event.preventDefault();

    if (nombreForm.value === "") {
      mensajeForm.textContent = "Debe ingresar un nombre.";
    } else {
      mensajeForm.textContent = "Formulario enviado correctamente.";
    }
  }

  formulario.addEventListener("submit", validarFormulario);
</script>
```

addEventListener vincula el evento "submit" del formulario a la función validar. El ejemplo es didáctico.

Javascript: eventos comunes

- click: se ejecuta cuando el usuario hace clic sobre un elemento.
- dblclick: se ejecuta cuando el usuario hace doble clic.
- mouseover: se ejecuta cuando el mouse entra sobre un elemento.
- mouseout: se ejecuta cuando el mouse sale de un elemento.
- mousemove: se ejecuta cuando el mouse se mueve sobre un elemento.
- mousedown: se ejecuta cuando se presiona un botón del mouse.
- mouseup: se ejecuta cuando se suelta un botón del mouse.
- keydown: se ejecuta cuando el usuario presiona una tecla.
- keyup: se ejecuta cuando el usuario suelta una tecla.
- input: se ejecuta cada vez que cambia el valor de un campo, por ejemplo mientras se escribe.
- change: se ejecuta cuando cambia el valor de un campo y el cambio queda confirmado.
- submit: se ejecuta cuando se intenta enviar un formulario.
- reset: se ejecuta cuando se reinicia un formulario.
- focus: se ejecuta cuando un elemento recibe el foco.
- blur: se ejecuta cuando un elemento pierde el foco.
- select: se ejecuta cuando el usuario selecciona texto dentro de un campo.
- load: se ejecuta cuando terminó de cargar la página o un recurso.
- DOMContentLoaded: se ejecuta cuando el HTML ya fue cargado y el DOM está disponible.
- resize: se ejecuta cuando cambia el tamaño de la ventana.
- scroll: se ejecuta cuando el usuario desplaza la página.
- beforeunload: se ejecuta antes de abandonar o recargar la página.
- copy: se ejecuta cuando el usuario copia contenido.
- cut: se ejecuta cuando el usuario corta contenido.
- paste: se ejecuta cuando el usuario pega contenido.

[Referencias]

- <https://www.w3schools.com/js/default.asp>
- <http://www.ecma-international.org/publications/standards/Ecma-262.htm>